Fpt University

TRƯỜNG ĐẠI HỌC FPT

Data Storage

Contents

- Shared Preferences
- Internal/External Files
- Sqlite
- Room database

Shared Preferences

- Shared Preferences allows activities and applications to keep preferences, in the form of key-value pairs
- Android stores Shared Preferences settings as XML file in shared_prefs folder under DATA/data/{application package} directory
- To get access to the preferences, we have APIs:
  - getPreferences() : used from within your Activity, to access activity-specific preferences
  - getSharedPreferences() : used from within your Activity (or other application Context), to access application-level preferences

Shared Preferences

- **Implement:**
  - **getSharedPreferences** (String PREFS_NAME, int mode)

    PREFS_NAME is the name of the file.

    mode is the operating mode.

Shared Preferences

- **Initialization**

```
SharedPreferences pref = getApplicationContext().getSharedPreferences("MyPref",
Context.MODE_PRIVATE);
Editor editor = pref.edit();
```

- **Storing Data**

```
editor.putBoolean("key_name", true); // Storing boolean - true/false
editor.putString("key_name", "string value"); // Storing string
editor.putInt("key_name", "int value"); // Storing integer
editor.putFloat("key_name", "float value"); // Storing float
editor.putLong("key_name", "long value"); // Storing long

editor.commit(); // commit changes
```

Shared Preferences

- **Retrieving Data**

```
pref.getString("key_name", null); // getting String
pref.getInt("key_name", null); // getting Integer
pref.getFloat("key_name", null); // getting Float
pref.getLong("key_name", null); // getting Long
pref.getBoolean("key_name", null); // getting boolean
```

- **Clearing or Deleting Data:**
  - remove(“key_name”) is used to delete that particular value.
  - clear

```
editor.remove("name"); // will delete key name
editor.remove("email"); // will delete key email

editor.commit(); // commit changes
```

```
editor.clear();
editor.commit(); // commit changes
```

File Storage

- External storage -- Public directories
- Internal storage -- Private directories for just your app

Apps can browse the directory structure

Structure and operations similar to Linux and java.io

File Storage

- **Internal storage is best when**
  - you want to be sure that neither the user nor other apps can access your files
- **External storage is best for files that**
  - don't require access restrictions
  - you want to share with other apps
  - you allow the user to access with a computer

File Storage

- **Internal Storage**
  - Always available
  - Uses device's filesystem
  - Only your app can access files, unless explicitly set to be readable or writable
  - On app uninstall, system removes all app's files from internal storage

File Storage

- **External storage**
  - Not always available, can be removed
  - Uses device's file system or physically external storage like SD card
  - World-readable, so any app can read
  - On uninstall, system does not remove files private to app

Using Internal Storage

- **An activity has methods you can call to read/write files:**
  - getFilesDir() - returns internal directory for your app
  - getCacheDir() - returns a "temp" directory for scrap files
  - getResources().openRawResource(R.raw.id) - read an input file from res/raw/
  - openFileInput("name", mode) - opens a file for reading
  - openFileOutput("name", mode) - opens a file for writing

Using Internal Storage – Example 1

```
// read a file, and put its contents into a TextView
// (assumes hello.txt file exists in res/raw/ directory)
Scanner scan = new Scanner(
        getResources().openRawResource(R.raw.hello));
String allText = "";   // read entire file
while (scan.hasNextLine()) {
    String line = scan.nextLine();
    allText += line;
}
myTextView.setText(allText);
scan.close();
```

Using Internal Storage – Example 2

```
// write a short text file to the internal storage
PrintStream output = new PrintStream
        openFileOutput("out.txt", MODE_PRIVATE));
output.println("Hello, world!");
output.println("How are you?");
output.close();
...
// read the same file, and put its contents into a TextView
Scanner scan = new Scanner(
        openFileInput("out.txt", MODE_PRIVATE));
String allText = "";    // read entire file
while (scan.hasNextLine()) {
    String line = scan.nextLine();
    allText += line;
}
myTextView.setText(allText);
scan.close();
```

Using External Storage

- **Must explicitly request permission**

```
<uses-permission
   android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission
    android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

- **Methods to read/write external storage:**
  - getExternalFilesDir("name") - returns "private" external directory for your app with the given name
  - Environment.getExternalStoragePublicDirectory(name) - returns public directory for common files like photos, music, etc.

External Storage – Example

```
// write short data to app-specific external storage
File outDir = getExternalFilesDir(null);   // root dir
File outFile = new File(outDir, "example.txt");
PrintStream output = new PrintStream(outFile);
output.println("Hello, world!");
output.close();

// read list of pictures in external storage
File picsDir =
        Environment.getExternalStoragePublicDirectory(
                Environment.DIRECTORY_PICTURES);
for (File file : picsDir.listFiles()) {
    ...
}
```

External Storage - Checking if storage is available

```
/* Checks if external storage is available
 * for reading and writing */
public boolean isExternalStorageWritable() {
    return Environment.MEDIA_MOUNTED.equals(
            Environment.getExternalStorageState());
}

/* Checks if external storage is available
 * for reading */
public boolean isExternalStorageReadable() {
    return isExternalStorageWritable() ||
            Environment.MEDIA_MOUNTED_READ_ONLY.equals(
                Environment.getExternalStorageState());
}
```

Assignment

- In LoginActivity, using SharedPreference to handle:
  - If user has not login app, show LoginActivity
  - If user logined app, show HomeActivity

Sqlite database

- Store data in tables of rows and columns (spreadsheet…)
- Field = intersection of a row and column
- Fields contain data, references to other fields, or references to other tables
- Rows are identified by unique IDs
- Column names are unique per table

Sqlite database

- **Table**

WORD_LIST_TABLE		
_id	**word**	**definition**
1	"alpha"	"first letter"
2	"beta"	"second letter"
3	"alpha"	"particle"

SQL basic operations

- Insert rows
- Delete rows
- Update values in rows
- Retrieve rows that meet given criteria

SQL Query

- SELECT word, description
  FROM WORD_LIST_TABLE
  WHERE word="alpha"

Generic

- SELECT columns
  FROM table
  WHERE column="value"

SELECT columns FROM table

- **SELECT columns**
  - Select the columns to return
  - Use * to return all columns

- **FROM table—specify the table from which to get results**

WHERE column="value"

- WHERE—keyword for conditions that have to be met
- column="value"—the condition that has to be met
  - common operators: =, LIKE, <, >

AND, ORDER BY, LIMIT

SELECT _id FROM WORD_LIST_TABLE WHERE word="alpha" AND definition LIKE "%art%" ORDER BY word DESC LIMIT 1

- AND, OR—connect multiple conditions with logic operators
- ORDER BY—omit for default order, or ASC for ascending, DESC for descending
- LIMIT—get a limited number of results

Sample queries

1	SELECT * FROM WORD_LIST_TABLE	Get the whole table
2	SELECT word, definition FROM WORD_LIST_TABLE WHERE _id > 2	Returns [["alpha", "particle"]]

More sample queries

3	SELECT _id FROM WORD_LIST_TABLE WHERE word="alpha" AND definition LIKE "%art%"	Return id of word alpha with substring "art" in definition [["3"]]
4	SELECT * FROM WORD_LIST_TABLE ORDER BY word DESC LIMIT 1	Sort in reverse and get first item. Sorting is by the first column (_id) [["3","alpha","particle"]]

Last sample query

5	SELECT * FROM WORD_LIST_TABLE LIMIT 2,1	Returns 1 item starting at position 2. Position counting starts at 1 (not zero!). Returns [["2","beta","second letter"]]

rawQuery()

```
String query = "SELECT * FROM WORD_LIST_TABLE";
rawQuery(query, null);

query = "SELECT word, definition FROM WORD_LIST_TABLE
WHERE _id> ? ";

String[] selectionArgs = new String[]{"2"}
rawQuery(query, selectionArgs);
```

query()

SELECT * FROM WORD_LIST_TABLE WHERE word="alpha" ORDER BY word ASC LIMIT 2,1; Returns: [["alpha", "particle"]]	String table = "WORD_LIST_TABLE" String[] columns = new String[]{"*"}; String selection = "word = ?" String[] selectionArgs = new String[]{"alpha"}; String groupBy = null; String having = null; String orderBy = "word ASC" String limit = "2,1" query(table, columns, selection, selectionArgs, groupBy, having, orderBy, limit);

Cursors

Queries always return a Cursor object

Cursor is an object interface that provides random read-write access to the result set returned by a database query

⇒ Think of it as a pointer to table rows

Using sqlite

- SQLiteDatabase db = openOrCreateDatabase("name", MODE_PRIVATE, null);
  db.execSQL("SQL query");
- Methods:
  - – db.beginTransaction(), db.endTransaction()
  - – db.delete("table", "whereClause" , args)
  - – db.deleteDatabase(file)
  - – db.insert("table", null, values)
  - – db.query(...)
  - – db.rawQuery("SQL query", args)
  - – db.replace("table", null, values)
  - – db.update("table", values, "whereClause", args)

Using sqlite

- ContentValues can be optionally used as a level of abstraction for statements like INSERT, UPDATE, REPLACE

```
ContentValues cvalues = new ContentValues();
cvalues.put("columnName1", value1);
cvalues.put("columnName2", value2);
...
db.insert("tableName", null, cvalues);

db.execSQL("INSERT INTO tableName (" + columnName1 + ", "
+ columnName2 + ") VALUES (" + value1 + ", " + value2 + ")");
```

Using sqlite

- **SQLiteOpenHelper** class provides the functionality to use the SQLite database

Constructor	Description
SQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version)	creates an object for creating, opening and managing the database.
SQLiteOpenHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version, DatabaseErrorHandler errorHandler)	creates an object for creating, opening and managing the database. It specifies the error handler.

Using sqlite

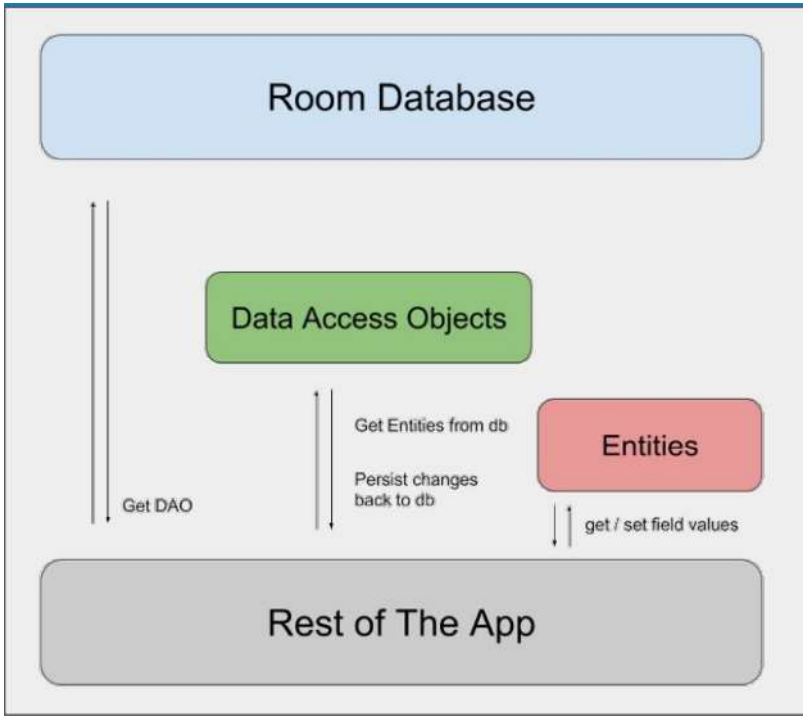
Method	Description
public abstract void onCreate(SQLiteDatabase db)	called only once when database is created for the first time.
public abstract void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)	called when database needs to be upgraded.
public synchronized void close ()	closes the database object.
public void onDowngrade(SQLiteDatabase db, int oldVersion, int newVersion)	called when database needs to be downgraded.

Using sqlite

Method	Description
void execSQL(String sql)	executes the sql query not select query.
long insert(String table, String nullColumnHack, ContentValues values)	inserts a record on the database. The table specifies the table name, nullColumnHack doesn't allow completely null values. If second argument is null, android will store null values if values are empty. The third argument specifies the values to be stored.
int update(String table, ContentValues values, String whereClause, String[] whereArgs)	updates a row.
Cursor query(String table, String[] columns, String selection, String[] selectionArgs, String groupBy, String having, String orderBy)	returns a cursor over the resultset.

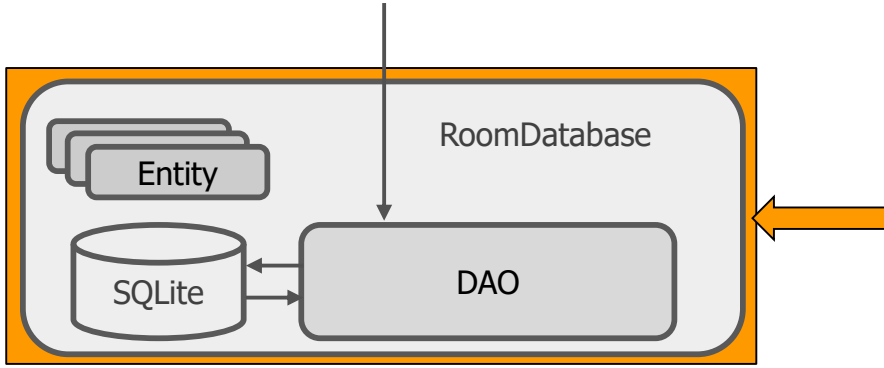
Room database

- Room is a robust SQL object mapping library
- Generates SQLite Android code



Room database

- Room works with DAO and Entities
- Entities define the database schema
- DAO provides methods to access database



Using Room database

- Add library
- Create Entity
- Create DAO
- Create Database

Using Room database

- **Add library**

```
// Room components
implementation "androidx.room:room-runtime:$rootProject.roomVersion"
annotationProcessor "androidx.room:room-compiler:$rootProject.roomVersion"
androidTestImplementation "androidx.room:room-testing:$rootProject.roomVersion"
```

Using Room database

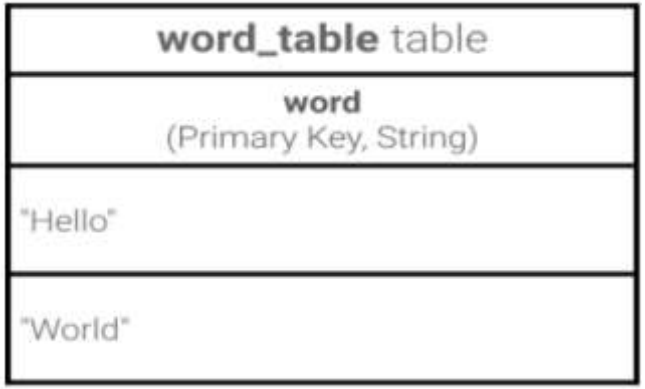
- **Create Entity**

```
@Entity(tableName = "word_table")
public class Word {

    @PrimaryKey
    @NonNull
    @ColumnInfo(name = "word")
    private String mWord;

    public Word(@NonNull String word) {this.mWord = word;}

    public String getWord(){return this.mWord;}
}
```



Using Room database

- **Create DAO**

```
@Dao
public interface WordDao {

    // allowing the insert of the same word multiple times by passing a
    // conflict resolution strategy
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    void insert(Word word);

    @Query("DELETE FROM word_table")
    void deleteAll();

    @Query("SELECT * FROM word_table ORDER BY word ASC")
    List<Word> getAlphabetizedWords();
}
```

Using Room database

- **Create database**
  - Create public abstract class extending `RoomDatabase`
  - Annotate as `@Database`
  - Declare entities for database schema and set version number

@Database(entities = {Word.class}, version = 1)

public abstract class WordRoomDatabase extends RoomDatabase

Using Room database

```
@Database(entities = {Word.class}, version = 1)
public abstract class WordRoomDatabase
                    extends RoomDatabase {

  public abstract WordDao wordDao();

  private static WordRoomDatabase INSTANCE;

  // ... create instance here
}
```

Entity defines DB schema

DAO for database

Create database as singleton instance

Using Room database

- **Use Database builder**
  - Use Room's database builder to create the database
  - Create DB as singleton instance

```
private static WordRoomDatabase INSTANCE;
INSTANCE = Room.databaseBuilder(...)
    .build();
```

Room database creation example

```
static WordRoomDatabase getDatabase(final Context context) {
  if (INSTANCE == null) {
    synchronized (WordRoomDatabase.class) {
      if (INSTANCE == null)    {
        INSTANCE = Room.databaseBuilder(
                        context.getApplicationContext(),
                        WordRoomDatabase.class, "word_database")
                        .build();
      }}}
  return INSTANCE;
```

Classwork

- Create a database in sqlite, named “goods.db”
- Create a table, named “product”

Id	ProductName	Product Description	ProductPrice
1	Laptop	Abc…	20.000.000
2	Smartphone	Xyz…	15.000.00
3	Headphone	Kzz…	1000.000

- Load data into the recyclerview (slot 10)
- Create Insert/Update/Delete function for Product screen

References

- https://developer.android.com/codelabs/android-room-with-a-view#0
- https://google-developer-training.github.io/android-developer-fundamentals-course-concepts-v2/